

PYTHON PROGRAMMING & PYTHON LIBRARIES

Complete syllabus-based study material for students and internship learners

MODULE 1	Fundamentals of Python — 11 Units
MODULE 2	Python Libraries — NumPy, Pandas, Matplotlib, Seaborn & Plotly
FORMAT	Concept → Explanation → Example → Practice → Checkpoint

Read each topic, type the examples yourself, run them in Jupyter, change one value, observe the output, and complete the checkpoint before moving ahead.

Important: The supplied material is study material rather than only a topic list. The syllabus has been expanded into explanations, examples, practice-oriented notes and checkpoints.

How This Course Is Structured

Stage	What the student should be able to do
1. Foundations	Write, run and debug basic Python programs; understand variables, operators and built-in functions.
2. Logic & repetition	Make decisions with conditions and automate repetitive work with loops.
3. Reusable code	Create functions, pass arguments, return results and use flexible arguments.
4. Core data structures	Work confidently with strings, lists, tuples, sets and dictionaries.
5. Numerical computing	Use NumPy arrays, vectorized operations, indexing, statistics and linear algebra.
6. Data analysis	Use Pandas to load, clean, select, transform, summarize and visualize tabular data.
7. Visualization	Create static and interactive charts with Matplotlib, Seaborn and Plotly.

Study habit: For every code example: predict the output → run it → change one value → explain why the output changed → save the exercise in your notebook.

PYTHON PROGRAMMING& PYTHON LIBRARIES

Complete syllabus-based study material for students and internship learners

MODULE 1 Fundamentals of Python — 11 Units

MODULE 2 Python Libraries — NumPy, Pandas, Matplotlib, Seaborn & Plotly

FORMAT Concept → Explanation → Example → Practice → Checkpoint

HOW TO USE Read each topic, type the examples yourself, complete the practice tasks, then move to the unit checkpoint.

Important: This document is designed as study material, not just a topic list. The syllabus items supplied for the course are expanded into explanations, examples, and learning outcomes.

How This Course Is Structured

The course moves from Python fundamentals to numerical computing, data manipulation and visualization. Each unit builds on the previous one.

Stage What the student should be able to do

1. Foundations Write, run and debug basic Python programs; understand variables, operators and built-in functions.
2. Logic & repetition Make decisions with conditions and automate repetitive work with loops.
3. Reusable code Create functions, pass arguments, return results and use flexible arguments.
4. Core data structures Work confidently with strings, lists, tuples, sets and dictionaries.
5. Numerical computing Use NumPy arrays, vectorized operations, indexing, statistics and linear algebra.
6. Data analysis Use Pandas to load, clean, select, transform, summarize and visualize tabular data.
7. Visualization Create static and interactive charts with Matplotlib, Seaborn and Plotly.

Study habit

For every code example: (1) predict the output, (2) run it, (3) change one value, (4) explain why the output changed, and (5) save the exercise in a notebook.

MODULE 1

Fundamentals of Python

Goal: develop a strong programming foundation before moving into NumPy, Pandas and visualization.

UNIT 1

The Basics of Python Programming

Understanding Problem Solving

Problem solving in programming means converting a real requirement into a sequence of precise steps that a computer can execute. A useful process is: understand the problem, identify inputs and expected outputs, break the task into smaller steps, write an algorithm or pseudocode, implement it, test it, and improve it.

- Separate the problem from the programming language.

- Think about normal cases, boundary cases and invalid input.
- Use small steps rather than trying to write the complete solution at once.

Why Python

Python is a high-level, general-purpose language known for readable syntax and a large ecosystem. It is widely used for automation, data analysis, visualization, web development and machine learning. For beginners, its syntax lets them focus on problem-solving concepts rather than complex language mechanics.

- Readable and concise syntax.
- Large standard library and third-party ecosystem.
- Excellent support for notebooks and data-analysis libraries.

What is an IDE?

An Integrated Development Environment combines tools for writing, running and debugging code. Typical features include a code editor, terminal, project/file explorer, debugger, syntax highlighting and extensions. Jupyter Notebook is especially useful for learning and data analysis because code, output, notes and visualizations can live together.

- IDE: a complete programming workspace.
- Jupyter Notebook: interactive cells for code and explanation.
- Use an environment that makes running and debugging code easy.

Installation: Anaconda & Jupyter Notebook

Anaconda provides Python plus commonly used data-science packages and environment management. After installation, Jupyter Notebook can be launched from Anaconda Navigator or a terminal command. The important learning objective is understanding where code runs and how notebooks are organized.

- Create a dedicated course folder.
- Keep notebooks named by unit and exercise.
- Restart and re-run cells when results become confusing.

PYTHON EXAMPLE

Open Anaconda Navigator → launch Jupyter Notebook → create a Python notebook → run a cell with: `print('Hello, P`

Write Your First Python Program

The first program should demonstrate output and the basic execution cycle. Python executes statements in sequence. `print()` sends a value to the output area.

- Understand statements and indentation.
- Use comments to document intent.
- Run code in small pieces and inspect output.

PYTHON EXAMPLE

```
print('Hello, Python!')
name = 'Student'
print('Welcome,', name)
```

UNIT CHECKPOINT — After UNIT 1, the student should be able to explain the concepts, reproduce the examples without copying, and solve 3–5 new exercises using the same ideas.

UNIT 2

Variables and Operators in Python

Variables

A variable is a name that refers to a value. Python determines the type at runtime, so you can assign an integer, string or another object without declaring the type first. Good variable names describe the data they hold.

- Use descriptive names such as `total_sales` rather than `x`.
- Reassigning a variable changes what it refers to.
- Use `type(value)` when you need to inspect a value's type.

PYTHON EXAMPLE

```
age = 21
name = 'Asha'
score = 87.5
```

Arithmetic Operators

Arithmetic operators perform numeric calculations: `+` addition, `-` subtraction, `*` multiplication, `/` true division, `//` floor division, `%` remainder and `**` exponentiation. Floor division and modulo are especially useful for grouping and extracting remainders.

- Distinguish `/` from `//`.
- Use `%` for remainder checks such as even/odd logic.
- Use parentheses when expression order is important.

PYTHON EXAMPLE

```
a, b = 17, 5
print(a + b)
print(a / b)
print(a // b)
print(a % b)
print(a ** 2)
```

Assignment Operators

The basic assignment operator `=` stores a value. Compound operators such as `+=`, `-=`, `*=`, `/=` and `%=` update an existing variable using its current value.

- Assignment is not the same as mathematical equality.
- Compound assignment makes accumulator code concise.

PYTHON EXAMPLE

```
total = 100
total += 25
print(total)
```

Boolean & Logical Operators

Boolean values are `True` and `False`. Logical operators `and`, `or` and `not` combine or invert conditions. They are fundamental for filtering data and building decision rules.

- `and` requires both conditions to be true.
- `or` needs at least one true condition.
- `not` reverses a Boolean result.

PYTHON EXAMPLE

```
age = 22
```

```
has_id = True
print(age >= 18 and has_id)
```

Comparison Operators

Comparison operators return Boolean results. Common operators are ==, !=, >, <, >= and <=. They are used in ifstatements, loops and filtering expressions.

- Use == for comparison and = for assignment.
- Comparisons can be combined with logical operators.

PYTHON EXAMPLE

```
marks = 72
print(marks >= 40)
print(marks == 100)
```

Bitwise Operators

Bitwise operators work on the binary representation of integers. &, |, ^, ~, << and >> perform bit-level operations. They are useful for flags, masks and low-level data handling, although they are less common in beginner data analysis.

- Learn the binary representation of small integers.
- Do not confuse & with logical and.
- Understand shifts as bit movement rather than ordinary arithmetic.

PYTHON EXAMPLE

```
a, b = 6, 3
print(a & b)
print(a | b)
print(a ^ b)
print(a << 1)
print(a >> 1)
```

UNIT CHECKPOINT — After UNIT 2, the student should be able to explain the concepts, reproduce the examples without copying, and solve 3–5 new exercises using the same ideas.

UNIT 3

Built-in Functions of Python

round() and divmod()

round() rounds a number to a chosen number of decimal places. divmod(a, b) returns a pair containing quotient and remainder, which is useful when a calculation naturally needs both results.

- round(x, n) keeps n decimal places.
- divmod() returns (quotient, remainder).

PYTHON EXAMPLE

```
price = 19.9876
print(round(price, 2))
print(divmod(17, 5))
```

isinstance() and pow()

isinstance(value, type) checks whether an object belongs to a type or one of several types. pow() calculates powers and also supports a third argument for modular exponentiation.

- Use isinstance() when validation depends on type.
- pow(a,b) is equivalent to a ** b for ordinary powers.

PYTHON EXAMPLE

```
print(isinstance(10, int))
print(pow(2, 5))
print(pow(2, 5, 7))
```

input()

input() reads text typed by the user and returns a string. Convert the returned value when numeric input is required.

- Always remember that input() returns str.
- Validate or convert input before calculations.

PYTHON EXAMPLE

```
name = input('Enter your name: ')
age = int(input('Enter age: '))
print(name, age)
```

int() and float()

int() converts compatible values to integers and float() converts compatible numeric strings or values to floating-point numbers. Conversion may fail when the text is not numeric.

- Understand conversion versus rounding.
- Use try/except later when robust input validation is needed.

PYTHON EXAMPLE

```
x = int('25')
y = float('3.14')
print(x + 5, y * 2)
```

len(), min(), max()

len() returns the number of items in supported sequences/collections. min() and max() return the smallest and largest values. They are basic tools for inspecting data.

- len() counts items, not necessarily characters unless the object is a string.
- min/max work with comparable values.

PYTHON EXAMPLE

```
scores = [72, 91, 65, 88]
print(len(scores))
print(min(scores), max(scores))
```

sorted()

sorted() creates a new sorted list from an iterable. It does not change the original list. It can use reverse=True and a keyfunction for custom ordering.

- sorted() returns a new list.
- Use key= for business-specific sorting.

PYTHON EXAMPLE

```
names = ['Zara', 'Amit', 'Neha']
print(sorted(names))
print(sorted(names, reverse=True))
```

help()

help() displays documentation for Python objects and functions. It is an important habit for becoming independent from memorized notes.

- Use help() to inspect parameters and behavior.
- Combine documentation with small experiments.

PYTHON EXAMPLE

```
help(round)
help(str.split)
```

UNIT CHECKPOINT — After UNIT 3, the student should be able to explain the concepts, reproduce the examples without copying, and solve 3–5 new exercises using the same ideas.

UNIT 4

Control Flow Statements

if Condition

An if statement executes a block only when its condition is true. Python uses indentation to define the block.

- Indent consistently.
- Conditions should evaluate to True or False.

PYTHON EXAMPLE

```
temperature = 31
if temperature > 30:
    print('Hot day')
```

if-else Condition

if-else provides two paths: one when the condition is true and another when it is false.

- Ensure both possible outcomes are considered.
- Keep each branch focused on one responsibility.

PYTHON EXAMPLE

```
marks = 56
if marks >= 40:
    print('Pass')
else:
    print('Fail')
```

if-elif-else Condition

Use elif when multiple mutually exclusive conditions are possible. Python tests conditions from top to bottom and executes the first matching branch.

- Order conditions carefully.
- Put the broad fallback in else.

PYTHON EXAMPLE

```
score = 82
if score >= 90:
    grade = 'A+'
elif score >= 75:
    grade = 'A'
elif score >= 60:
    grade = 'B'
else:
    grade = 'C'
print(grade)
```

Nested if

A nested if is an if statement inside another conditional block. It is useful when the second decision only matters after the first condition succeeds.

- Avoid deeply nested logic when a combined condition is clearer.

- Indentation communicates program structure.

PYTHON EXAMPLE

```
age = 20
has_id = True
if age >= 18:
    if has_id:

print('Entry allowed')
else:
    print('ID required')
```

UNIT CHECKPOINT — After UNIT 4, the student should be able to explain the concepts, reproduce the examples without copying, and solve 3–5 new exercises using the same ideas.

UNIT 5

Concepts of Loops

while Loop

A while loop repeats while a condition remains true. It is useful when the number of iterations is not known in advance. Always ensure the loop condition can eventually become false.

- Initialize the control variable.
- Update it inside the loop.
- Watch for infinite loops.

PYTHON EXAMPLE

```
count = 1
while count <= 5:
    print(count)
    count += 1
```

break and continue

break stops the nearest loop immediately. continue skips the remainder of the current iteration and moves to the next iteration.

- Use break for a deliberate early exit.
- Use continue to skip selected cases.

PYTHON EXAMPLE

```
for n in range(1, 8):
    if n == 5:
        break
    print(n)
```

for Loop

A for loop iterates over the items of an iterable such as a string, list, tuple, set, dictionary or range. It is the common choice when processing every item in a collection.

- Think 'for each item'.
- Use enumerate() when you need both index and value.

PYTHON EXAMPLE

```
for name in ['Asha', 'Ravi', 'Meera']:
    print(name)
```


range() Function

range() generates a sequence of integers without creating a full list in memory. It supports start, stop and step; the stopvalue is excluded.

- range(5) gives 0–4.
- The stop value is exclusive.
- Negative steps can count backwards.

PYTHON EXAMPLE

```
for i in range(2, 10, 2):
    print(i)
```

Nested Loops

A nested loop places one loop inside another. It is useful for grids, combinations and two-dimensional data, but the number of iterations can grow quickly.

- Track which variable belongs to which loop.
- Consider whether nested loops can be simplified for large data.

PYTHON EXAMPLE

```
for row in range(3):
    for col in range(4):
        print(row, col)
```

UNIT CHECKPOINT — After UNIT 5, the student should be able to explain the concepts, reproduce the examples without copying, and solve 3–5 new exercises using the same ideas.

UNIT 6

User-Defined Functions in Python

Introduction & Docstring

A function packages reusable logic under a name. A docstring describes what the function does and can be displayed by help(). Functions reduce duplication and make programs easier to test.

- Use verbs or clear action-oriented names.
- Keep a function focused on one job.
- Document inputs, output and important assumptions.

PYTHON EXAMPLE

```
def area(radius):
    """Return the area of a circle."""
    return 3.14159 * radius ** 2
```

Input Arguments

Parameters receive values supplied when the function is called. Arguments can be positional or keyword-based.

- Match arguments to parameters.
- Keyword arguments improve readability when many parameters exist.

PYTHON EXAMPLE

```
def greet(name, city):
    return f'Hello {name} from {city}'
print(greet('Asha', 'Pune'))
```

Order of Input Arguments

Python supports positional parameters, default parameters, variable positional arguments and keyword arguments. The order matters because required positional parameters must be arranged correctly relative to defaults and flexible arguments.

- Required parameters generally come before parameters with defaults.
- Learn the calling rules before using advanced signatures.

PYTHON EXAMPLE

```
def report(name, score=0):
    return f'{name}: {score}'
print(report('Asha'))
```

Return Statement

return sends a result back to the caller and ends the function. Returning a value is different from printing it: returned values can be stored, reused and passed into other functions.

- Prefer return when the caller needs the result.
- A function can return multiple values using a tuple.

PYTHON EXAMPLE

```
def add(a, b):
    return a + b
result = add(10, 5)
print(result)
```

Variable Number of Arguments: *args and **kwargs

*args collects extra positional arguments into a tuple. **kwargs collects extra keyword arguments into a dictionary. They are useful for flexible reusable functions.

- *args is tuple-like; **kwargs is dictionary-like.
- Use these features when flexibility is genuinely useful.

PYTHON EXAMPLE

```
def total(*args):
    return sum(args)
```

```
def profile(**kwargs):
    return kwargs
```

```
print(total(10, 20, 30))
print(profile(name='Asha', city='Pune'))
```

Lambda Function

A lambda is a small anonymous function containing a single expression. It is commonly used as a key function or short transformation.

- Use lambda for short expressions.
- Use def for complex, reusable logic.

PYTHON EXAMPLE

```
students = [('Asha', 82), ('Ravi', 91)]
print(sorted(students, key=lambda x: x[1], reverse=True))
```

UNIT CHECKPOINT — After UNIT 6, the student should be able to explain the concepts, reproduce the examples without copying, and solve 3–5 new exercises using the same ideas.

UNIT 7

Python Strings

Introduction to String

A string is an immutable sequence of characters. Strings can contain letters, numbers, spaces and symbols. Because strings are immutable, methods return new strings rather than changing the original.

- Use single or double quotes consistently.
- Remember that indexing starts at 0.

PYTHON EXAMPLE

```
text = 'Python for Data Analysis'
print(text)
```

Indexing & Slicing

Indexing retrieves one character. Slicing extracts a range using start:stop:step. Negative indexes count from the end.

- Stop index is excluded.
- Use negative indexes for right-to-left access.

PYTHON EXAMPLE

```
text = 'PYTHON'
print(text[0])
print(text[-1])
print(text[1:5])
print(text[::-1])
```

strip(), lower(), upper()

strip() removes leading/trailing whitespace (and optionally selected characters). lower() and upper() normalize case, which is useful during data cleaning.

- Normalization is important before comparisons.
- strip() does not remove spaces in the middle.

PYTHON EXAMPLE

```
value = ' Python '
print(value.strip())
print(value.lower())
print(value.upper())
```

replace(), split(), capitalize()

replace() substitutes text, split() divides a string into a list, and capitalize() makes the first character uppercase while lowercasing the rest.

- split() is central to simple text parsing.
- replace() returns a new string.

PYTHON EXAMPLE

```
text = 'data-analysis-python'
print(text.replace('-', ' '))
print(text.split('-'))
print('python'.capitalize())
```

title(), swapcase(), find(), index()

title() applies title casing, swapcase() flips letter case, find() returns the first matching position or -1, while index() raises an error if the substring is absent.

- Use find() when absence is expected and should not raise an error.
- index() is useful when absence should be treated as an error.

PYTHON EXAMPLE

```
text = 'python data'
print(text.title())
print(text.swapcase())
print(text.find('data'))
print(text.index('data'))
```

isalnum(), isalpha(), isdigit()

These predicates validate character content. isalnum() checks letters/numbers, isalpha() checks letters only, and isdigit() checks digit characters.

- These methods return Boolean values.
- They are useful in input validation.

PYTHON EXAMPLE

```
print('Python3'.isalnum())
print('Python'.isalpha())
print('12345'.isdigit())
```

isupper(), islower(), istitle(), startswith(), endswith()

These methods test capitalization and prefixes/suffixes. They are useful for validation and classification of text.

- Use tuples with startswith/endswith when several prefixes/suffixes are acceptable.

PYTHON EXAMPLE

```
filename = 'Report_2026.CSV'
print(filename.endswith('.CSV'))
print('PYTHON'.isupper())
print('Python Basics'.istitle())
```

Multiline Strings & Substrings

Triple quotes allow multiline strings. The in operator checks whether a substring exists inside another string.

- Use multiline strings for documentation or structured text.
- Substring checks are case-sensitive unless you normalize case.

PYTHON EXAMPLE

```
message = '''Line one
Line two
Line three'''
print('Line two' in message)
```

Converting String into Another Data Type

Strings containing valid numeric text can be converted using int() or float(). Boolean conversion requires care because bool('False') is True since the string is non-empty.

- Validate text before conversion in real applications.
- Be careful with empty strings and non-numeric characters.

PYTHON EXAMPLE

```
qty = int('12')
price = float('19.50')
print(qty * price)
```

Loops in String

A string is iterable, so a for loop can process each character. This is useful for counting characters, validating content or building transformations.

- Combine loops with conditions for character-level processing.
- For large text tasks, specialized string tools can be more efficient.

PYTHON EXAMPLE

```
text = 'DATA'
for ch in text:
    print(ch)
```

UNIT CHECKPOINT — After UNIT 7, the student should be able to explain the concepts, reproduce the examples without copying, and solve 3–5 new exercises using the same ideas.

UNIT 8

Python List

Introduction, Indexing & Slicing

A list is an ordered, mutable collection that can hold multiple values. Indexing accesses one element and slicing extracts a range.

- Lists can contain mixed types, although homogeneous data is often easier to work with.
- List positions start at 0.

PYTHON EXAMPLE

```
nums = [10, 20, 30, 40]
print(nums[1])
print(nums[1:3])
```

append(), extend(), insert()

append() adds one item at the end. extend() adds each item from another iterable. insert() adds an item at a chosen position.

- append adds one object; extend adds multiple elements.
- insert can shift later indexes.

PYTHON EXAMPLE

```
items = ['A', 'B']
items.append('C')
items.extend(['D', 'E'])
items.insert(1, 'X')
print(items)
```

remove(), pop(), clear()

remove(value) deletes the first matching value. pop(index) removes and returns an element. clear() empties the list.

- pop() is useful when the removed value is needed.
- remove() raises ValueError if the value is absent.

PYTHON EXAMPLE

```
items = ['A', 'B', 'C']
items.remove('B')
last = items.pop()
print(items, last)
```

count(), max(), min(), reverse(), index(), copy()

count() counts occurrences. max/min find extremes. reverse() changes order in place. index() finds a matching position. copy() creates a shallow copy.

- reverse() modifies the original list.
- copy() avoids simple aliasing such as `b = a`.

PYTHON EXAMPLE

```
nums = [4, 2, 4, 9]
print(nums.count(4), max(nums), min(nums))
print(nums.index(9))
copy_nums = nums.copy()
```

Loops and Conditional Statements in Lists

Lists can be processed using for loops and filtered with if conditions. This forms the basis of list comprehensions and later Pandas filtering.

- Separate selection logic from transformation logic.
- Practice calculating totals, counts and filtered lists.

PYTHON EXAMPLE

```
scores = [35, 72, 81, 49]
for score in scores:
    if score >= 50:
        print(score)
```

UNIT CHECKPOINT — After UNIT 8, the student should be able to explain the concepts, reproduce the examples without copying, and solve 3–5 new exercises using the same ideas.

UNIT 9

Python Tuples

Introduction, Concatenation & Nesting

A tuple is an ordered, immutable collection. Tuples are useful for fixed groups of related values and can be concatenated or nested.

- Immutability means individual elements cannot be reassigned.
- Nested tuples can represent structured records.

PYTHON EXAMPLE

```
a = (1, 2)
b = (3, 4)
print(a + b)
nested = ((1, 2), (3, 4))
```

length, count, index, indexing & slicing

len() gives tuple length; count() counts occurrences; index() locates the first matching item. Indexing and slicing works similarly to strings and lists.

PYTHON EXAMPLE

```
t = ('A', 'B', 'A', 'C')
print(len(t), t.count('A'), t.index('C'))
print(t[1:3])
```

tuple() Constructor

tuple() converts an iterable into a tuple. It is useful when a sequence should become immutable or when a function expects tuple-like data.

PYTHON EXAMPLE

```
values = tuple([10, 20, 30])
print(values)
```

Duplicates, max(), min(), unpacking

Tuples may contain duplicates. max() and min() work when values are comparable. Tuple unpacking assigns elements to multiple variables in one statement.

- The number of unpacking variables normally must match the number of values.
- Use * during unpacking to collect remaining values.

PYTHON EXAMPLE

```
record = ('Asha', 92, 'Python')
name, score, subject = record
print(name, score, subject)
```

Conditional Statements and Loops in Tuple

Tuples can be inspected with conditions and iterated with for loops just like other iterables.

PYTHON EXAMPLE

```
t = (12, 7, 19, 4)
for x in t:
    if x > 10:
        print(x)
```

UNIT CHECKPOINT — After UNIT 9, the student should be able to explain the concepts, reproduce the examples without copying, and solve 3–5 new exercises using the same ideas.

UNIT 10

Python Sets

Introduction to Set

A set is an unordered collection of unique elements. Sets are useful for removing duplicates and performing mathematical set operations.

- Sets do not support positional indexing.
- Use a set when uniqueness matters.

PYTHON EXAMPLE

```
skills = {'Python', 'SQL', 'Python'}
print(skills)
```

add(), update(), remove(), discard(), pop(), clear(), del, union

add() inserts one element; update() adds multiple elements; remove() deletes an element and errors if absent; discard() does not error when absent; pop() removes an arbitrary element; clear() empties the set; del removes the set variable; union combines sets.

PYTHON EXAMPLE

```
a = {1, 2, 3}
a.add(4)
a.update([5, 6])
b = {3, 6, 7}
print(a.union(b))
```

intersection_update(), intersection(), difference_update(), difference()

Intersection returns common elements. Difference returns elements present in the first set but not the second. The *_update forms modify the set in place.

PYTHON EXAMPLE

```
a = {1, 2, 3}
b = {2, 3, 4}
print(a.intersection(b))
print(a.difference(b))
```

symmetric_difference_update(), symmetric_difference()

Symmetric difference returns elements belonging to either set but not both. The update version changes the original set.

PYTHON EXAMPLE

```
a = {1, 2, 3}
b = {3, 4, 5}
print(a.symmetric_difference(b))
```

isdisjoint(), issuperset(), issubset()

These methods compare relationships between sets. isdisjoint checks for no common elements; issubset checks whether all elements are contained; issuperset checks whether the set contains all elements of another set.

PYTHON EXAMPLE

```
a = {1, 2, 3}
b = {1, 2}
print(b.issubset(a))
print(a.issuperset(b))
```

set(), max(), min(), copy(), loops

set() creates an empty set or converts an iterable. max/min can be used when set elements are comparable. copy() creates a shallow copy. Sets can be iterated, although order should not be relied upon.

PYTHON EXAMPLE

```
values = set([5, 2, 5, 9])
print(values, min(values), max(values))
for x in values:
    print(x)
```

UNIT CHECKPOINT — After UNIT 10, the student should be able to explain the concepts, reproduce the examples without copying, and solve 3–5 new exercises using the same ideas.

UNIT 11

Python Dictionary

Introduction to Dictionary

A dictionary stores key-value pairs. Keys identify values and must be hashable. Dictionaries are central to JSON-like data, configuration, lookups and structured records.

- Use meaningful keys.
- Access with [] when a missing key should be treated as an error; get() is safer when absence is possible.

PYTHON EXAMPLE

```
student = {'name': 'Asha', 'score': 91}
print(student['name'])
```

len(), dict(), fromkeys(), setdefault()

len() counts key-value pairs. dict() constructs dictionaries. fromkeys() creates a dictionary with a shared default value. setdefault() returns an existing value or inserts a default when the key is missing.

PYTHON EXAMPLE

```
keys = ['name', 'city']
d = dict.fromkeys(keys, 'Unknown')
d.setdefault('age', 20)
print(d)
```

get(), keys(), values(), items(), update()

get() retrieves safely. keys(), values() and items() provide views for iteration. update() adds or replaces key-value pairs.

PYTHON EXAMPLE

```
d = {'name': 'Asha', 'score': 90}
print(d.get('score'))
print(list(d.keys()))
print(list(d.items()))
d.update({'score': 95, 'city': 'Pune'})
```

pop(), popitem(), del, clear(), copy()

pop(key) removes and returns a value. popitem() removes and returns the last inserted key-value pair. del removes a selected key or variable. clear() empties the dictionary. copy() creates a shallow copy.

PYTHON EXAMPLE

```
d = {'a': 1, 'b': 2, 'c': 3}
value = d.pop('b')
print(value, d)
d.clear()
```

Nested Dictionaries and Loops

A nested dictionary stores dictionaries inside a dictionary. This structure is common for hierarchical records. Loops can process keys, values or complete key-value pairs.

- Practice accessing multiple levels.
- Use items() when both key and value are needed.

PYTHON EXAMPLE

```
students = {
    'S01': {'name': 'Asha', 'score': 91},
    'S02': {'name': 'Ravi', 'score': 84}
}
for sid, info in students.items():
    print(sid, info['name'], info['score'])
```

UNIT CHECKPOINT — After UNIT 11, the student should be able to explain the concepts, reproduce the examples without copying, and solve 3–5 new exercises using the same ideas.

MODULE 2

Python Libraries

Goal: move from core Python into the tools commonly used for numerical computing, data analysis and visualization.

UNIT 1

NumPy

Introduction to NumPy

NumPy (Numerical Python) provides fast array-based computation. Its central object is the ndarray, which stores values in a structured multidimensional array and supports vectorized operations.

- Understand why arrays are useful for numerical data.
- Import using the conventional alias np.

PYTHON EXAMPLE

```
import numpy as np
arr = np.array([10, 20, 30])
print(arr)
```

ndarray Object & Data Types

An ndarray can have one or more dimensions and a defined dtype. Common dtypes include integer, floating point and Boolean types. Arrays generally work most efficiently when their values have compatible types.

PYTHON EXAMPLE

```
arr = np.array([1, 2, 3], dtype=np.int64)
print(arr.dtype)
print(arr.ndim)
```

Array Attributes

Important attributes include shape, ndim, size, dtype and itemsize. They describe the structure and storage characteristics of an array.

- shape tells the size of each dimension.
- size is the total number of elements.

PYTHON EXAMPLE

```
arr = np.arange(12).reshape(3, 4)
print(arr.shape)
print(arr.ndim)
print(arr.size)
```

Array Creation Routines

NumPy provides convenient constructors such as zeros(), ones(), full(), eye(), empty(), arange() and linspace(). These are useful for initializing arrays without manually writing every value.

PYTHON EXAMPLE

```
print(np.zeros((2, 3)))
print(np.ones(4))
print(np.linspace(0, 1, 5))
```

Array From Existing Data

`np.array()` converts compatible sequences into arrays. `asarray()` is useful when you want an array representation while avoiding an unnecessary copy in cases where the input is already an ndarray.

PYTHON EXAMPLE

```
data = [[1, 2], [3, 4]]
a = np.asarray(data)
print(a)
```

Arrays From Numerical Ranges

`arange()` creates values with a step, while `linspace()` creates a specified number of evenly spaced values between endpoints. `linspace` is often preferable when the number of points is known.

PYTHON EXAMPLE

```
print(np.arange(0, 10, 2))
print(np.linspace(0, 10, 6))
```

Indexing & Slicing

NumPy supports one-dimensional and multidimensional indexing. Slices can select rows, columns or blocks. Unlike ordinary Python lists, array slicing is deeply integrated with multidimensional data.

PYTHON EXAMPLE

```
a = np.arange(12).reshape(3, 4)
print(a[1, 2])
print(a[:, 1])
print(a[0:2, 1:3])
```

Advanced Indexing

Advanced indexing includes integer-array indexing and Boolean masking. It lets you select non-contiguous elements or filter data using conditions.

- Boolean masks are central to numerical filtering.
- Use masks to express conditions without explicit loops.

PYTHON EXAMPLE

```
a = np.array([10, 20, 30, 40])
print(a[[0, 2]])
print(a[a > 20])
```

Broadcasting

Broadcasting lets NumPy perform operations on arrays with compatible shapes without manually repeating data. It is one of the main reasons vectorized NumPy code is concise and fast.

- Learn shape compatibility.
- Broadcasting does not mean every shape can be combined.

PYTHON EXAMPLE

```
a = np.array([[1, 2], [3, 4]])
b = np.array([10, 20])
print(a + b)
```

Iterating Over Arrays

Arrays can be iterated with Python loops, but vectorized operations are generally preferred for numerical work. `np.nditer()` can provide a systematic way to traverse elements.

PYTHON EXAMPLE

```
a = np.array([[1, 2], [3, 4]])
for row in a:
    print(row)
```

Array Manipulation

Functions such as `reshape()`, `flatten()`, `ravel()`, `transpose()`, `concatenate()`, `stack()`, `split()`, `resize()` and `squeeze()` change array structure or combine arrays.

PYTHON EXAMPLE

```
a = np.arange(6)
print(a.reshape(2, 3))
print(a.reshape(2, 3).T)
```

Binary Operators

NumPy supports element-wise binary operations such as `+`, `-`, `*`, `/`, `//`, `%`, `**`, comparisons and logical-style array operations. The result is normally another array.

PYTHON EXAMPLE

```
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])
print(a * b)
print(a < 3)
```

String Functions

NumPy's character/string helpers can perform vectorized text operations on arrays of strings. They are useful when simple string transformations need to be applied element-wise.

PYTHON EXAMPLE

```
names = np.array(['asha', 'ravi'])
print(np.char.upper(names))
print(np.char.capitalize(names))
```

Mathematical & Arithmetic Functions

NumPy provides universal functions for common calculations such as `sqrt`, `exp`, `log`, `sin`, `cos`, `abs`, `add`, `subtract`, `multiply` and `divide`. These operate efficiently across arrays.

PYTHON EXAMPLE

```
a = np.array([1, 4, 9])
print(np.sqrt(a))
print(np.log(a))
```

Statistical Functions

Functions such as `mean`, `median`, `std`, `var`, `sum`, `min`, `max`, `percentile` and `average` summarize numerical data. The `axis` parameter allows calculations by row or column.

- Always understand which axis you are summarizing.
- Check missing-value behavior when moving to real datasets.

PYTHON EXAMPLE

```
a = np.array([[10, 20], [30, 40]])
print(np.mean(a))
print(np.mean(a, axis=0))
```

Sort, Search & Counting Functions

`sort()`, `argsort()`, search-related utilities, `unique()` and counting functions help order and inspect data. `np.unique()` is especially useful for identifying distinct values and their frequencies.

PYTHON EXAMPLE

```
a = np.array([3, 1, 3, 2])
print(np.sort(a))
print(np.unique(a, return_counts=True))
```

Byte Swapping

Byte order matters when raw binary data is moved between systems. `byteswap()` changes byte order representation. This is an advanced topic mainly relevant to binary data and interoperability.

PYTHON EXAMPLE

```
a = np.array([1, 2, 3], dtype=np.int32)
print(a.byteswap())
```

Copies & Views

A view shares underlying data with another array, while a copy owns independent data. Understanding this distinction prevents unexpected changes to source arrays.

- Use `copy()` when independent data is required.
- Views can be memory-efficient but need careful handling.

PYTHON EXAMPLE

```
a = np.array([1, 2, 3])
b = a.copy()
c = a.view()
b[0] = 99
print(a, b)
```

Matrix Library & Linear Algebra

NumPy includes matrix-related operations and the `linalg` module for linear algebra tasks such as matrix multiplication, determinants, inverses, solving systems, eigenvalues and decompositions.

- Prefer regular `ndarray` objects for most modern NumPy work.
- Use `@` for matrix multiplication when appropriate.

PYTHON EXAMPLE

```
A = np.array([[2., 1.], [1., 3.]])
b = np.array([5., 7.])
x = np.linalg.solve(A, b)
print(x)
```

UNIT CHECKPOINT — Create one small notebook for UNIT 1: load or create data, demonstrate the main operations, and write 3 observations in plain language.

UNIT 2

Pandas

Introduction to Pandas

Pandas is a Python library for working with labeled tabular and time-series data. Its main structures are `Series` and `DataFrame`.

PYTHON EXAMPLE

```
import pandas as pd
df = pd.DataFrame({'name': ['Asha', 'Ravi'], 'score': [91, 84]})
print(df)
```

Reading Data From Different File Systems

Pandas can load data from common formats such as CSV, Excel and JSON, and can connect to databases through appropriate interfaces. Reading data is the starting point for a reproducible analysis pipeline.

- Inspect the file after loading.
- Check column names, types and missing values immediately.

PYTHON EXAMPLE

```
df = pd.read_csv('sales.csv')
# df = pd.read_excel('sales.xlsx')
# df = pd.read_json('sales.json')
```

DataFrame

A DataFrame is a two-dimensional labeled table with rows and columns. Each column behaves like a Series and can have its own data type.

PYTHON EXAMPLE

```
df = pd.DataFrame({'Product': ['A', 'B', 'C'], 'Sales': [120, 150, 90]})
print(df.head())
```

DataFrame Operations & Basic Functionality

Core inspection tools include head(), tail(), info(), describe(), shape, columns, dtypes and value_counts(). These provide a quick profile before deeper analysis.

- Profile before transforming.
- Use describe() for numeric summaries and value_counts() for categorical distributions.

PYTHON EXAMPLE

```
print(df.head())
print(df.shape)
print(df.info())
print(df.describe())
```

Re-indexing

Reindexing changes or aligns row/column labels. It can introduce missing values when requested labels do not exist.

PYTHON EXAMPLE

```
df2 = df.reindex([2, 1, 0])
print(df2)
```

Iteration

DataFrames can be iterated over columns, rows or tuples. iterrows() is easy to understand but often slower than vectorized operations. itertuples() can be faster for row-wise iteration.

- Prefer vectorized Pandas operations over row loops when possible.
- Use iteration when the operation genuinely needs row-by-row logic.

PYTHON EXAMPLE

```
for col in df.columns:
    print(col)
```

Sorting

sort_values() orders rows by one or more columns. sort_index() orders by labels. ascending and inplace/copy behaviors should be understood before using them in pipelines.

PYTHON EXAMPLE

```
print(df.sort_values('Sales', ascending=False))
```

Working With Text Data

The `.str` accessor provides vectorized string operations such as `lower()`, `upper()`, `strip()`, `contains()`, `replace()`, `split()`, `startswith()` and `extract()`. It is essential for cleaning text columns.

PYTHON EXAMPLE

```
df['Product'] = df['Product'].str.strip().str.upper()
print(df)
```

Options & Customization

Pandas options control display and behavior, such as maximum rows/columns shown, precision and display width. Options are useful in notebooks but should not be confused with permanent data transformations.

PYTHON EXAMPLE

```
pd.set_option('display.max_columns', 20)
print(df)
```

Indexing & Selecting Data

`loc` selects by labels and conditions; `iloc` selects by integer positions. Boolean filtering is one of the most important Pandas skills.

- Use `loc` for label/condition logic.
- Use `iloc` for positional selection.

PYTHON EXAMPLE

```
print(df.loc[df['Sales'] > 100, ['Product', 'Sales']])
print(df.iloc[0:2, 0:2])
```

Statistical Functions

Pandas supports `sum`, `mean`, `median`, `min`, `max`, `std`, `count`, `quantile` and aggregation functions. These can be combined with `groupby` for business summaries.

PYTHON EXAMPLE

```
print(df['Sales'].mean())
print(df['Sales'].agg(['min', 'max', 'mean']))
```

Window Functions

Rolling, expanding and exponentially weighted calculations help analyze trends over ordered observations. They are useful for moving averages and cumulative metrics.

- Sort by the correct time/order column before applying rolling calculations.
- Understand the window size and missing values at the beginning.

PYTHON EXAMPLE

```
df['Rolling3'] = df['Sales'].rolling(3).mean()
print(df)
```

Date Functionality

Pandas datetime tools parse dates and expose components such as year, month, day and weekday. Date-aware data enables time-series analysis and resampling.

PYTHON EXAMPLE

```
df['Date'] = pd.to_datetime(df['Date'])
df['Month'] = df['Date'].dt.month
```

Time Delta

Timedelta represents a duration rather than a calendar date. It can be used to calculate differences between dates and add/subtract durations.

PYTHON EXAMPLE

```
start = pd.Timestamp('2026-01-01')
end = pd.Timestamp('2026-01-10')
print(end - start)
```

Categorical Data

Categorical dtype stores repeated labels efficiently and can make the meaning of categories explicit. It is useful for dimensions such as region, grade or customer segment.

PYTHON EXAMPLE

```
df['Region'] = df['Region'].astype('category')
print(df['Region'].dtype)
```

Visualization in Pandas

Pandas plotting provides a convenient interface over Matplotlib for quick charts. It is useful for exploration before creating a more polished visualization.

PYTHON EXAMPLE

```
df['Sales'].plot(kind='bar', title='Sales by Product')
```

Pandas Tools

Pandas includes tools for combining and transforming data: `merge()`, `join()`, `concat()`, `pivot_table()`, `melt()`, `crosstab()`, `groupby()` and related functions. These are the building blocks of practical data preparation.

- Choose `merge` for relational joins.
- Use `concat` for stacking compatible objects.
- Use `pivot/melt` to reshape analytical tables.

PYTHON EXAMPLE

```
summary = df.groupby('Region', as_index=False)['Sales'].sum()
print(summary)
```

UNIT CHECKPOINT — Create one small notebook for UNIT 2: load or create data, demonstrate the main operations, and write 3 observations in plain language.

UNIT 3

Matplotlib

Introduction to Matplotlib

Matplotlib is a foundational Python visualization library. It provides fine control over figures, axes, labels, annotations, legends and layout.

PYTHON EXAMPLE

```
import matplotlib.pyplot as plt
plt.plot([1,2,3], [10,20,15])
plt.show()
```

Bar Chart

Bar charts compare discrete categories. Use clear category labels and an appropriate axis scale.

PYTHON EXAMPLE

```
plt.bar(['A','B','C'], [10,15,8])
plt.title('Category Sales')
plt.xlabel('Category')
plt.ylabel('Sales')
plt.show()
```

Scatter Plot

Scatter plots show the relationship between two numerical variables. They help reveal trends, clusters and potential outliers.

- Use scatter plots when both axes represent numeric variables.
- Add transparency or grouping when many points overlap.

PYTHON EXAMPLE

```
plt.scatter([1,2,3,4], [2,4,3,7])
plt.xlabel('Hours')
plt.ylabel('Score')
plt.show()
```

UNIT CHECKPOINT — Create one small notebook for UNIT 3: load or create data, demonstrate the main operations, and write 3 observations in plain language.

UNIT 4

Seaborn

Introduction to Seaborn

Seaborn is a statistical visualization library built on Matplotlib. It offers high-level functions and works naturally with DataFrames, making exploratory analysis easier.

PYTHON EXAMPLE

```
import seaborn as sns
sns.set_theme()
sns.lineplot(data=df, x='Date', y='Sales')
```

Heatmap

A heatmap represents values with a color intensity grid. It is commonly used for correlation matrices, pivot-style summaries and pattern detection.

PYTHON EXAMPLE

```
sns.heatmap(df.corr(numeric_only=True), annot=True)
plt.show()
```

Count Plot

A count plot displays the number of observations in each category. It is useful for understanding category frequency.

PYTHON EXAMPLE

```
sns.countplot(data=df, x='Region')
plt.xticks(rotation=45)
plt.show()
```

Violin Plot

A violin plot combines distribution shape with summary information and is useful for comparing numerical distributions across categories.

PYTHON EXAMPLE

```
sns.violinplot(data=df, x='Region', y='Sales')
plt.show()
```

Pair Plot

pairplot creates a grid of relationships between several numerical variables and their distributions. It is useful for quick multivariate exploration.

PYTHON EXAMPLE

```
sns.pairplot(df.select_dtypes('number'))
plt.show()
```

Bar Graph, Histogram & Pie Chart

Bar plots compare categories; histograms show the distribution of a numerical variable; pie charts show part-to-whole relationships and should be used sparingly when categories are few and meaningful.

- Choose the chart based on the analytical question, not decoration.
- Histograms require numeric data.
- Pie charts become difficult to read with many categories.

PYTHON EXAMPLE

```
sns.histplot(data=df, x='Sales', bins=10)
plt.show()
```

Stem Plot

A stem plot displays discrete numerical values using stems and markers. It can reveal the structure of small numerical datasets.

PYTHON EXAMPLE

```
plt.stem([1,2,3,4], [3,7,4,9])
plt.show()
```

Box vs Whisker Plot

A box-and-whisker plot summarizes a distribution using quartiles, median and whiskers, making it useful for comparing spread and spotting potential outliers.

PYTHON EXAMPLE

```
sns.boxplot(data=df, x='Region', y='Sales')
plt.show()
```

Area & Stack Plot

Area charts show magnitude over an ordered axis. Stack plots show how multiple components contribute to a total overtime.

PYTHON EXAMPLE

```
x = [1,2,3,4]
y1 = [2,3,4,5]
y2 = [1,2,2,3]
plt.stackplot(x, y1, y2)
plt.show()
```

Step Plot

A step plot changes values at discrete points rather than drawing a smooth continuous transition. It is useful for schedules, states and piecewise values.

PYTHON EXAMPLE

```
plt.step([1,2,3,4], [10,10,20,15], where='mid')
plt.show()
```

fill_between

fill_between shades the area between curves or between a curve and a baseline. It can communicate ranges, confidencebands or thresholds.

PYTHON EXAMPLE

```
x = [1,2,3,4]
y = [2,3,2,4]
plt.plot(x, y)
plt.fill_between(x, y, 0, alpha=0.2)
plt.show()
```

subplot

Subplots place multiple axes inside one figure. They are useful when several related views need to be compared without opening separate figures.

PYTHON EXAMPLE

```
fig, axes = plt.subplots(1, 2, figsize=(8,3))
axes[0].plot([1,2,3])
axes[1].bar(['A','B'], [4,7])
plt.tight_layout()
plt.show()
```

savefig

savefig exports a figure to an image or vector format. Choose an appropriate resolution and bounding box for reports.

PYTHON EXAMPLE

```
plt.plot([1,2,3])
plt.savefig('chart.png', dpi=200, bbox_inches='tight')
```

Axis & Matplotlib Plot Text

Axes can be customized with labels, limits, ticks and scales. Text elements include title, xlabel, ylabel, legend, annotations and figure text.

- Every chart should communicate what the axes mean.
- Use annotations when a specific point is important.

PYTHON EXAMPLE

```
plt.plot([1,2,3], [2,5,4], label='Sales')
plt.title('Monthly Sales')
plt.xlabel('Month')
plt.ylabel('Revenue')
plt.legend()
plt.show()
```

Strip Plot

A strip plot shows individual observations across categories and is useful when you want to see the raw data points rather than only a summary.

PYTHON EXAMPLE

```
sns.stripplot(data=df, x='Region', y='Sales', jitter=True)
plt.show()
```

Box Plot

Seaborn's boxplot is a statistical view of distribution by category and is useful for comparing median, spread and possible outliers.

PYTHON EXAMPLE

```
sns.boxplot(data=df, x='Region', y='Sales')
plt.show()
```

Factor Plot / Modern Categorical Plot

factorplot was a Seaborn interface used for categorical plotting in older versions. In current Seaborn, catplot is the general figure-level categorical interface. The learning objective is understanding categorical visualization and choosing an appropriate kind.

- For new code, prefer catplot rather than legacy factorplot.
- Choose kind='bar', 'box', 'violin', 'strip', etc. based on the question.

PYTHON EXAMPLE

```
sns.catplot(data=df, x='Region', y='Sales', kind='box')
plt.show()
```

Cat Plot

catplot is a figure-level interface for categorical plots. It can create bar, count, box, violin, strip and related categorical views.

PYTHON EXAMPLE

```
sns.catplot(data=df, x='Region', y='Sales', kind='bar')
plt.show()
```

Styling Plot

Styling improves readability through figure size, labels, tick formatting, themes, context and layout. Good styling should support the message rather than distract from it.

PYTHON EXAMPLE

```
sns.set_theme(style='whitegrid', context='notebook')
ax = sns.barplot(data=df, x='Region', y='Sales')
ax.set_title('Sales by Region')
plt.tight_layout()
```

Facet Grid

Facet grids repeat a chart across subsets of the data, allowing comparisons between groups while keeping the visual structure consistent.

PYTHON EXAMPLE

```
g = sns.FacetGrid(df, col='Region')
g.map_dataframe(sns.histplot, x='Sales')
plt.show()
```

Line Plot

Line plots show change over an ordered variable, especially time. They are useful for trends and comparisons across multiple series.

PYTHON EXAMPLE

```
sns.lineplot(data=df, x='Date', y='Sales', hue='Region')
plt.show()
```

Histogram Plot

Histograms divide numeric values into bins to show distribution. The bin count affects the level of detail, so choose it thoughtfully.

PYTHON EXAMPLE

```
sns.histplot(data=df, x='Sales', bins=12, kde=True)
plt.show()
```

Bar Plot

Seaborn's barplot summarizes a numeric variable by category and, by default, estimates an aggregate such as the mean. Always know what statistic the bar represents.

PYTHON EXAMPLE

```
sns.barplot(data=df, x='Region', y='Sales')
plt.show()
```

Scatter Plot

Scatter plots compare two numeric variables and can use hue, size or style to reveal additional dimensions.

PYTHON EXAMPLE

```
sns.scatterplot(data=df, x='MarketingSpend', y='Sales', hue='Region')
plt.show()
```

UNIT CHECKPOINT — Create one small notebook for UNIT 4: load or create data, demonstrate the main operations, and write 3 observations in plain language.

UNIT 5

Plotly

Introduction to Plotly

Plotly is an interactive visualization library. Charts can support hover information, zooming, filtering and rich web-friendly presentation. It is useful when the audience needs to explore the data rather than only view a static image.

PYTHON EXAMPLE

```
import plotly.express as px
fig = px.line(df, x='Date', y='Sales', color='Region')
fig.show()
```

Bubble Charts

A bubble chart is a scatter plot where marker size represents an additional numeric variable. Use size carefully so visual area does not exaggerate differences.

PYTHON EXAMPLE

```
fig = px.scatter(df, x='Sales', y='Profit', size='Customers', color='Region',
                hover_name='Product')
fig.show()
```

Bar Charts

Plotly bar charts compare categories interactively and can display hover details without overcrowding the visible chart.

PYTHON EXAMPLE

```
fig = px.bar(df, x='Product', y='Sales', color='Region')
fig.show()
```

Pie Charts

Plotly pie charts show part-to-whole relationships with interactive labels and hover details. Keep the number of slices small for readability.

PYTHON EXAMPLE

```
fig = px.pie(df, names='Region', values='Sales', title='Sales Share by Region')
fig.show()
```

Creating Maps

Plotly supports geographic visualizations using latitude/longitude, choropleths and map layers. Map design should match the geographic question and use valid location data.

- Check coordinate quality before mapping.
- Use meaningful geographic boundaries or points.

PYTHON EXAMPLE

```
fig = px.scatter_map(df, lat='lat', lon='lon', size='Sales', hover_name='City', zoom=4)
fig.show()
```

Time Series

Plotly line charts are effective for interactive time-series exploration. Dates should be parsed correctly and sorted chronologically before plotting.

PYTHON EXAMPLE

```
df['Date'] = pd.to_datetime(df['Date'])
df = df.sort_values('Date')
fig = px.line(df, x='Date', y='Sales', color='Region', markers=True)
fig.show()
```

UNIT CHECKPOINT — Create one small notebook for UNIT 5: load or create data, demonstrate the main operations, and write 3 observations in plain language.

Practical Learning & Project Guide

From Syntax to Data Analysis

The fastest way to make the syllabus useful is to connect every concept to a small data task. The supplied roadmap emphasizes a repeatable analyst workflow: define the question, collect data, clean it, explore it, visualize it, explain the insight and recommend an action. This Python material can be practiced with the same mindset.

Step Student action

1. Define Write one measurable question, such as: Which region generated the highest revenue?
2. Collect Read CSV/Excel/JSON data with Pandas or create a small practice DataFrame.
3. Clean Inspect types, missing values, duplicates, text formatting and dates.
4. Explore Use Pandas summaries, NumPy statistics and grouped calculations.
5. Visualize Choose a chart that answers the question: bar, line, scatter, box, heatmap, etc.
6. Explain Write the finding in one or two sentences using numbers from the analysis.
7. Recommend State the practical next action and any limitation of the analysis.

Mini Project 1 — Student Performance Analysis

Create a DataFrame containing student name, course, score, attendance and completion status. Use Python fundamentals for validation and functions; Pandas for cleaning and grouping; NumPy for statistics; and Seaborn/Matplotlib for visualization.

Task Expected work

Data preparation Create or load the dataset; inspect columns and data types; handle missing values.

Analysis Calculate mean, median, minimum, maximum and pass rate. Compare performance by course.

Visualization Create a score distribution, course comparison and attendance-vs-score scatter plot.

Conclusion Write at least five insights and two recommendations.

Mini Project 2 — Sales Analysis

Use a sales dataset with date, product, region, quantity, sales and profit. The project should demonstrate the transition from Python syntax to practical data analysis.

Skill Suggested implementation

Python basics Functions for validation and reusable calculations.

NumPy Numerical summaries and array-based calculations.

Pandas Read data, clean text/dates, group by region/product, sort and reshape.

Matplotlib/Seaborn Static charts for trends, distributions and comparisons.

Plotly Interactive time-series or regional exploration.

Business output Top products, strongest regions, monthly trend, profit concern and recommended action.

Final Skill Checklist

- Write Python programs using variables, operators, conditions and loops.
- Use built-in functions confidently and explain what each one returns.
- Create functions with arguments, return values, defaults, *args and **kwargs.
- Manipulate strings, lists, tuples, sets and dictionaries.
- Create and reshape NumPy arrays and use indexing, broadcasting and statistics.
- Load and inspect data with Pandas; filter, sort, group, merge and reshape it.
- Work with text, dates, timedeltas, categorical data and rolling calculations.
- Build meaningful charts with Matplotlib and Seaborn.
- Create interactive charts, maps and time-series visuals with Plotly.
- Explain an analytical result in plain language and document assumptions.

STUDY RULE Do not learn Python by only reading. Type every example, change the inputs, predict the output, debug at least one error, and build a small project after each major unit.

Reference design note

This study material uses the supplied Data Analyst Roadmap as the visual direction for the navy/teal/gold palette and structured headings, while intentionally using a study-material layout rather than reproducing the roadmap's two-page design. The roadmap's learning philosophy also emphasizes practice, projects and explaining results, which is reflected in the checkpoints and projects here.

Python Visualization Lab

Runnable examples + chart interpretation

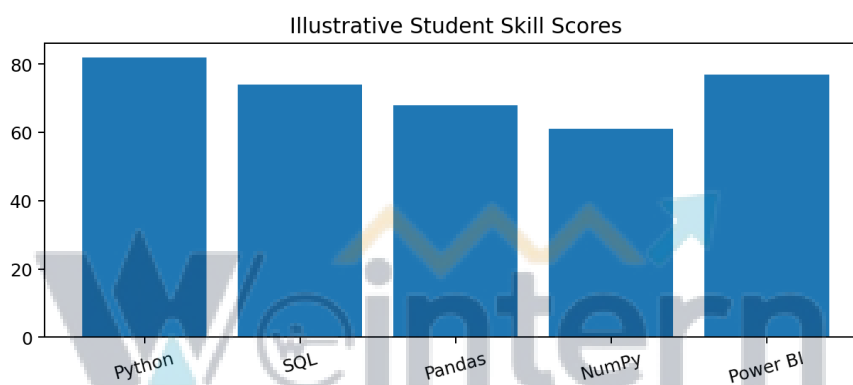
1. Matplotlib Bar Chart

Use a bar chart when the question compares discrete categories.

```
import matplotlib.pyplot as plt

skills = ["Python", "SQL", "Pandas", "NumPy", "Power BI"]
scores = [82, 74, 68, 61, 77]

plt.bar(skills, scores)
plt.title("Student Skill Scores")
plt.xlabel("Skill")
plt.ylabel("Score")
plt.ylim(0, 100)
plt.show()
```



2. Line Chart

Use a line chart when the x-axis is ordered, especially for time.

```
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = [120, 138, 132, 155, 171, 184]

plt.plot(months, sales, marker="o")
plt.title("Monthly Sales Trend")
plt.grid(True)
plt.show()
```

PRACTICE

Change one score and one month value, rerun both charts and write one sentence explaining the visual change.

Python Visualization Lab

Line charts for ordered trends

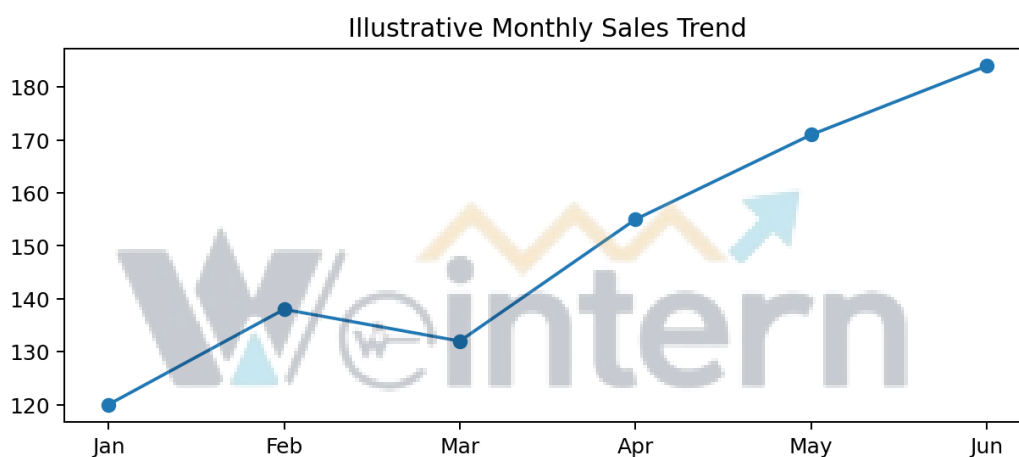
2. Matplotlib Line Chart

A line chart is useful when order matters, especially for monthly or daily measurements.

```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = [120, 138, 132, 155, 171, 184]

plt.plot(months, sales, marker="o")
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.grid(True)
plt.show()
```



Reading the trend

Look for direction, peaks, dips and changes in slope. Ask what business or process event could explain a large change.

- Do not treat a line chart as proof of causation.
- Sort dates before plotting time-series data.
- Use the same unit and time interval across the series.

Python Visualization Lab

Scatter plots for relationships

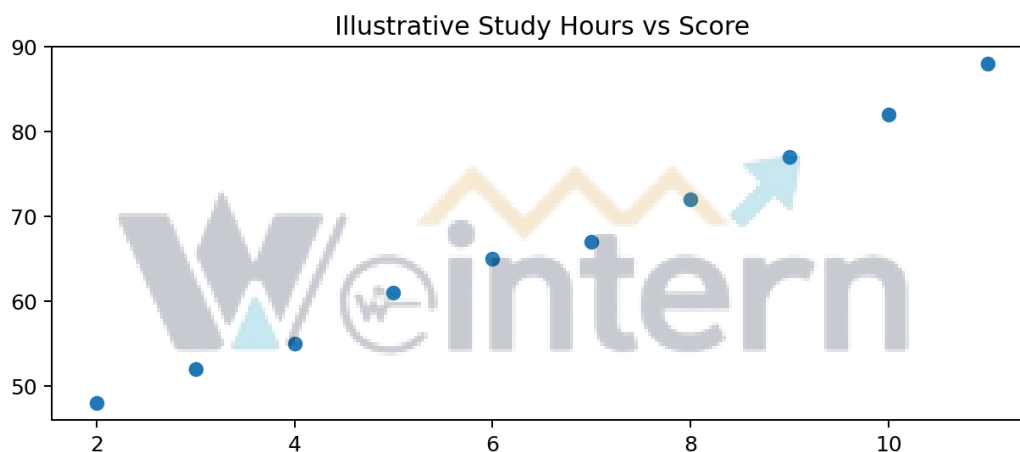
3. Scatter Plot: Study Hours vs Score

A scatter plot displays one point per observation and helps inspect the relationship between two numeric variables.

```
import matplotlib.pyplot as plt

study_hours = [2,3,4,5,6,7,8,9,10,11]
scores = [48,52,55,61,65,67,72,77,82,88]

plt.scatter(study_hours, scores)
plt.title("Study Hours vs Score")
plt.xlabel("Study hours")
plt.ylabel("Score")
plt.grid(True)
plt.show()
```



INTERPRETATION

The illustrative points rise as study hours increase. This shows association in the sample; it does not by itself establish causation.

Seaborn Lab

Statistical visualizations with DataFrames

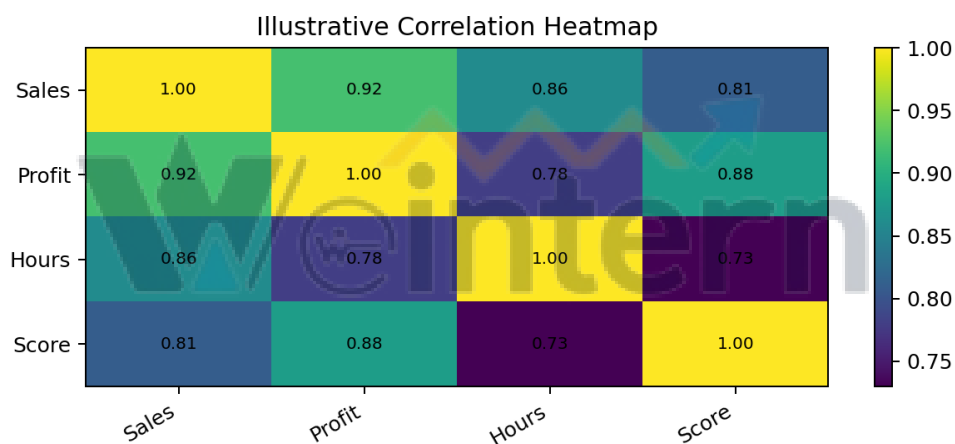
4. Heatmap + Count Plot

Seaborn provides higher-level statistical plots and integrates naturally with Pandas.

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

data = pd.DataFrame({
    "Sales": [100, 120, 150, 180],
    "Profit": [18, 22, 31, 38],
    "Hours": [2, 4, 6, 8],
    "Score": [50, 60, 72, 84]
})

sns.heatmap(data.corr(numeric_only=True), annot=True)
plt.title("Correlation Heatmap")
plt.show()
```



When to use a heatmap

Use a heatmap to scan a matrix for strong or weak values. A correlation heatmap is useful for exploratory analysis before modeling.

Distribution Analysis

Histogram, box plot and violin plot

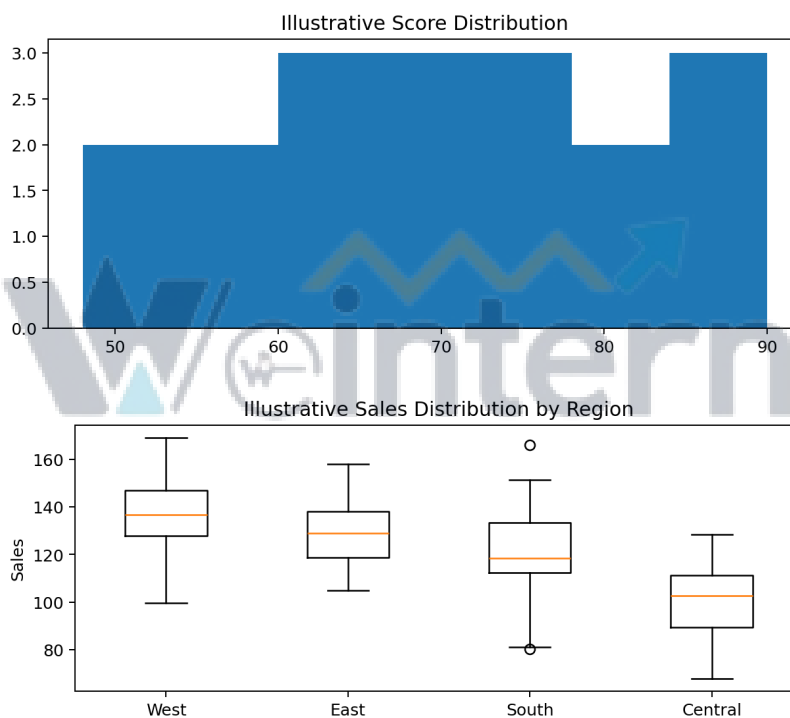
Choosing the right distribution chart

Histograms show frequency across numeric bins. Box plots summarize quartiles and possible outliers. Violin plots show distribution shape by category.

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.histplot(data=df, x="Sales", bins=10, kde=True)
plt.title("Sales Distribution")
plt.show()

sns.boxplot(data=df, x="Region", y="Sales")
plt.show()
```



INTERPRETATION

Use the histogram for overall distribution and the box plot for category-by-category spread and potential outliers.

Pandas Data Cleaning Lab

Load → inspect → clean → validate

A practical cleaning sequence

The source material emphasizes inspecting column names, types and missing values immediately after loading.

```
import pandas as pd

df = pd.read_csv("sales.csv")

print(df.head())
print(df.info())
print(df.isna().sum())

df["Product"] = df["Product"].str.strip().str.upper()
df["Date"] = pd.to_datetime(df["Date"], errors="coerce")

df = df.drop_duplicates()
df = df.dropna(subset=["Sales"])

print(df.describe())
```

Quality checks

Cleaning should be reproducible. Keep the raw data separate from the cleaned DataFrame and record important assumptions.

Check	Question	Example
Types	Are dates/numbers parsed cor	df.dtypes
Missing	Which fields are incomplete?	df.isna().sum()
Duplicates	Are repeated records expecte	df.duplicated().sum()
Text	Are labels consistently form	str.strip(), str.upper()
Range	Are values plausible?	df[Sales] .describe()

Pandas Analysis Lab

Filtering, grouping and sorting

From rows to business summaries

Filtering selects observations; groupby creates grouped summaries; sorting ranks the resulting data.

```
filtered = df[df["Sales"] > 110]

summary = (
    df.groupby("Region", as_index=False)["Sales"]
        .sum()
        .sort_values("Sales", ascending=False)
)

print(filtered)
print(summary)
```

Useful extensions

Add more measures with named aggregations and calculate ratios only after confirming the numerator and denominator are valid.

```
summary = (
    df.groupby("Region", as_index=False)
        .agg(
            Sales=("Sales", "sum"),
            Profit=("Profit", "sum")
        )
)

summary["Margin"] = summary["Profit"] / summary["Sales"]
print(summary)
```

PRACTICE

Change the filter threshold, add a region, calculate Profit Margin and identify the region with the highest sales.

NumPy Lab

Arrays, broadcasting, statistics and linear algebra

Array workflow

NumPy arrays make numerical operations concise and vectorized.

```
import numpy as np

a = np.arange(12).reshape(3, 4)
b = np.array([10, 20, 30, 40])

print(a)
print(a.shape, a.ndim)
print(a + b)           # broadcasting
print(a[a > 5])        # Boolean mask
print(np.mean(a, axis=0)) # column means
```

Linear algebra example

The supplied syllabus includes matrix operations and `np.linalg` for solving systems.

```
A = np.array([[2., 1.],
              [1., 3.]])
b = np.array([5., 7.])

x = np.linalg.solve(A, b)
```

CHECKPOINT

Explain `shape`, `ndim`, `broadcasting` and `Boolean masking` in your own words, then modify the array dimensions and predict the result.

Plotly Interactive Lab

Interactive charts, maps and time series

Interactive time-series chart

Plotly is useful when viewers need hover, zoom and interactive exploration.

```
import pandas as pd
import plotly.express as px

df["Date"] = pd.to_datetime(df["Date"])
df = df.sort_values("Date")

fig = px.line(
    df,
    x="Date",
    y="Sales",
    color="Region",
    markers=True,
    title="Sales Trend by Region"
)
fig.show()
```

Interactive bubble chart

A bubble chart uses marker size for an additional numeric variable.

```
fig = px.scatter(
    df,
    x="Sales",
    y="Profit",
    size="Customers",
    color="Region",
    hover_name="Product"
)
fig.show()
```

DESIGN RULE

Interactivity should answer a real exploration question. Do not add hover effects simply because the library supports them.

Python Analytics Dashboard

A compact dashboard built from Pandas + charts

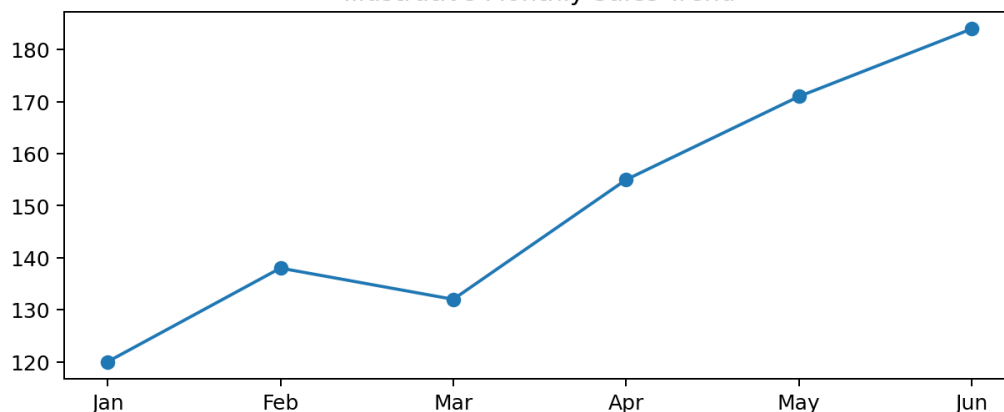
TOTAL SALES
798K

TOTAL PROFIT
52K

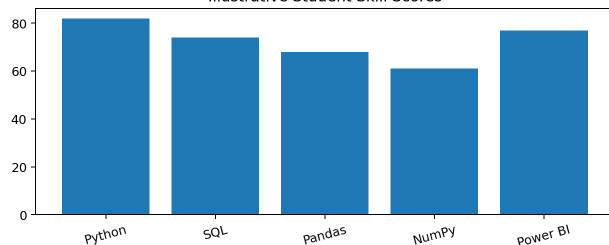
ORDERS
1,240

MARGIN
6.5%

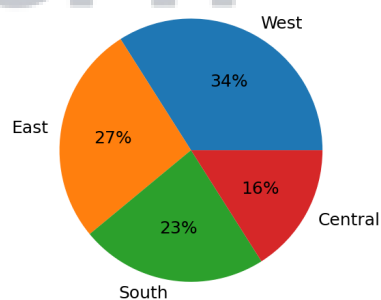
Illustrative Monthly Sales Trend



Illustrative Student Skill Scores



Illustrative Sales Share



DASHBOARD LOGIC

In a real notebook, calculate KPI values from the same cleaned DataFrame used by the charts. Keep the metric definitions documented.

Mini Project 1 — Student Performance

End-to-end Python project

Project question

Which course or skill area shows the strongest performance, and how is performance related to study effort?

Stage	Student task	Expected output
Define	Write 2 measurable questions	Question statement
Collect	Create/load CSV or DataFrame	Raw dataset
Clean	Check missing values and dup	Clean dataset
Analyze	Mean, median, min, max, pass	Summary table
Visualize	Bar + histogram + scatter	3 charts
Explain	Write 5 observations	Plain-language report
Recommend	Give 2 actions	Conclusion

Starter code

Use this as a scaffold, then modify it rather than submitting unchanged code.

```
summary = df.groupby("Course")["Score"].agg(
    ["mean", "median", "min", "max"]
).reset_index()

pass_rate = (df["Score"] >= 40).mean()
print(summary)
print("Pass rate:", pass_rate)
```

DELIVERABLE

Notebook + dataset + three charts + five insights + two recommendations.

Mini Project 2 — Sales Analysis

Clean, summarize and visualize business data

Project dataset

Use fields such as Date, Product, Region, Quantity, Sales and Profit. The source roadmap describes this project as a bridge from Python syntax to practical analysis.

Question	Python technique	Chart
Top products	groupby + sort_values	Bar
Strongest regions	groupby + aggregation	Bar/Pie
Monthly trend	datetime + groupby	Line
Profit concern	filter + ratio	Bar
Sales vs profit	column comparison	Scatter

Starter pipeline

```
df["Date"] = pd.to_datetime(df["Date"])
df["Margin"] = df["Profit"] / df["Sales"]

monthly = (
    df.groupby(df["Date"].dt.to_period("M"))["Sales"]
      .sum()
      .reset_index()
)

region = (
    df.groupby("Region", as_index=False)
      .agg(Sales=("Sales", "sum"), Profit=("Profit", "sum"))
      .sort_values("Sales", ascending=False)
)

print(region)
```

BUSINESS OUTPUT

Top products, strongest regions, monthly trend, profit concern and one recommended action.

Chart Selection Guide

Choose the visual based on the analytical question

Question	Recommended chart	Why
Compare categories	Bar	Clear ranking and comparison
Show change over time	Line	Order and trend are visible
Show a distribution	Histogram	Shows frequency across number
Compare distributions	Box / Violin	Shows spread and central tendency
Inspect a relationship	Scatter	One point per observation
Show matrix patterns	Heatmap	Color makes patterns visible
Show part-to-whole	Pie	Only when few meaningful parts
Explore interactively	Plotly chart	Hover, zoom and filtering

Common mistakes

A chart can be technically valid and still communicate poorly.

- Too many categories in a pie chart.
- Unsorted dates in a time-series line chart.
- Missing axis labels or units.
- Using decoration instead of an analytical message.
- Comparing values with incompatible scales.

Matplotlib Chart Anatomy

Figure, axes, labels, legend and annotations

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(["Jan", "Feb", "Mar", "Apr"], [120, 138, 132, 155],
        marker="o", label="Sales")

ax.set_title("Monthly Sales")
ax.set_xlabel("Month")
ax.set_ylabel("Revenue")
ax.legend()
ax.annotate("Peak", xy=("Apr", 155))
plt.tight_layout()
plt.savefig("sales.png", dpi=200, bbox_inches="tight")
plt.show()
```

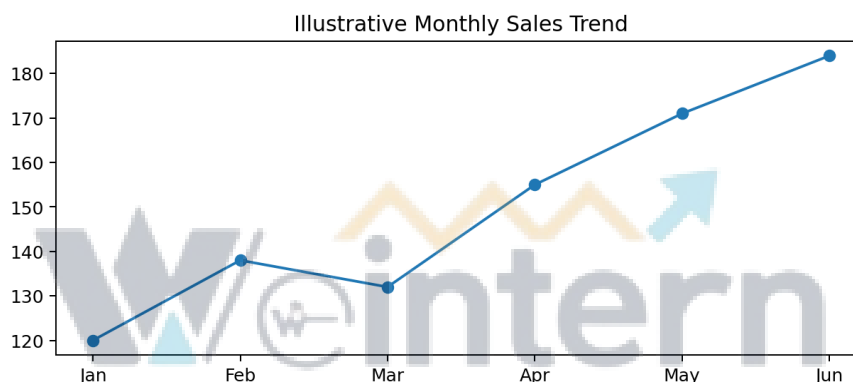


Chart quality checklist

Before publishing a chart, verify the title, axis labels, units, legend, readable ticks, figure size and export settings.

Element	Question
Title	Can a reader understand the
Axes	Are the variables and units
Legend	Is every series identifiable
Annotation	Does it highlight a meaningf
savefig	Is the exported image readab

Seaborn Categorical Lab

Count, bar, strip, box, violin and catplot

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(data=df, x="Region")
plt.xticks(rotation=45)
plt.show()

sns.catplot(
    data=df,
    x="Region",
    y="Sales",
    kind="box"
)
plt.show()
```

How the plots differ

Count plots count observations. Bar plots summarize a numeric variable by category. Strip plots expose individual observations. Box and violin plots focus on distributions.

Plot	Best question
Count	How many records are in each
Bar	What is the aggregate level
Strip	Where are the individual obs
Box	How do median and spread com
Violin	How does distribution shape
Catplot	How can I build a figure-lev

Plotly Maps & Time Series

Geographic and chronological exploration

Map example

Plotly can use latitude/longitude to create interactive geographic views. Validate coordinate quality before mapping.

```
import plotly.express as px

fig = px.scatter_map(
    df,
    lat="lat",
    lon="lon",
    size="Sales",
    hover_name="City",
    zoom=4,
    title="Sales Locations"
)
fig.show()
```

Time-series preparation

Dates should be parsed correctly and sorted chronologically.

```
df["Date"] = pd.to_datetime(df["Date"])
df = df.sort_values("Date")

fig = px.line(
    df, x="Date", y="Sales",
    color="Region", markers=True
)
fig.show()
```

VALIDATION

Before interpreting a map or time series, check missing coordinates, invalid dates, duplicate timestamps and inconsistent units.

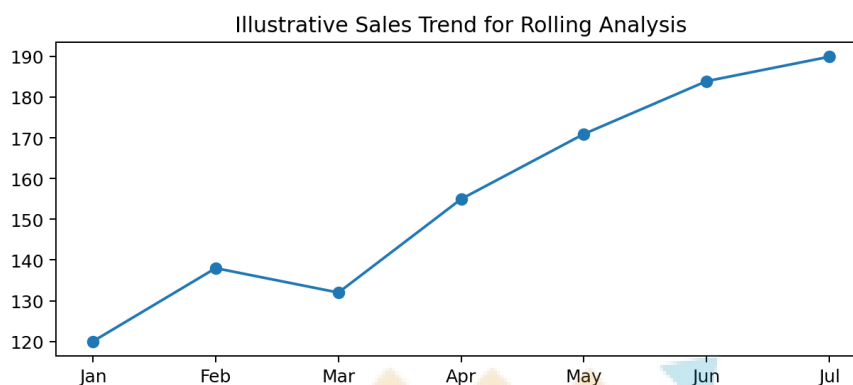
Pandas Time & Window Analysis

Dates, timedeltas and rolling calculations

```
df["Date"] = pd.to_datetime(df["Date"])
df = df.sort_values("Date")

df["Year"] = df["Date"].dt.year
df["Month"] = df["Date"].dt.month
df["Rolling3"] = df["Sales"].rolling(3).mean()

print(df[["Date", "Sales", "Rolling3"]])
```



Feature	Use
to_datetime	Parse dates
.dt.year / month / day	Extract components
Timedelta	Represent duration
rolling(n)	Moving-window calculation
expanding()	Cumulative calculation
ewm()	Exponentially weighted calcu

PRACTICE

Create a 5-row daily sales dataset and calculate a 3-day rolling average. Explain why the first two rows may be missing.

Final Capstone — Python Data Analyst Portfolio

A complete end-to-end workflow

Project brief

Choose Student Performance or Sales Analysis. Build one notebook that demonstrates Python fundamentals, NumPy, Pandas and visualization.

Section	Minimum requirement
Problem	One measurable business/lear
Data	CSV/Excel/JSON or documented
Cleaning	Types, missing values, dupli
Analysis	At least 3 grouped/statistic
Charts	Bar + line/histogram + scatt
Interactive	One Plotly chart
Insights	At least 5 plain-language ob
Recommendations	At least 2 actions
Documentation	Assumptions, limitations and

Notebook structure

Title → question → data dictionary → import → cleaning → exploration → analysis → visualization → insights → recommendations → limitations.

Deliverable	Done
Notebook (.ipynb)	■
Clean dataset	■
Three or more charts	■
Five insights	■
Two recommendations	■
README / assumptions	■

PORTFOLIO TIP

A strong project connects the question, analysis, visual evidence and recommendation. More code is not automatically better.

Final Assessment & Skill Checklist

Use this page as the course completion gate

Skill	Ready?	Evidence
Python basics	■	Program with variables and o
Control flow	■	Conditions + loops
Functions	■	Arguments, return, *args, **
Data structures	■	String/list/tuple/set/dict t
NumPy	■	Array, mask, broadcast, stat
Pandas	■	Load, clean, filter, group,
Matplotlib	■	Create and save a chart
Seaborn	■	Distribution/categorical cha
Plotly	■	Interactive chart or map
Analysis	■	Five insights + two recommen

Self-score

Give yourself 0 = not yet, 1 = with notes, 2 = independently. A total of 16+ indicates a strong foundation for the next project.

Area	Score (0-2)
Programming foundations	___
Functions & data structures	___
NumPy	___
Pandas	___
Visualization	___
Project communication	___
TOTAL	___ / 12

FINAL STUDY RULE

Do not learn Python by only reading. Type every example, change values, predict outputs, inspect errors and explain the result.